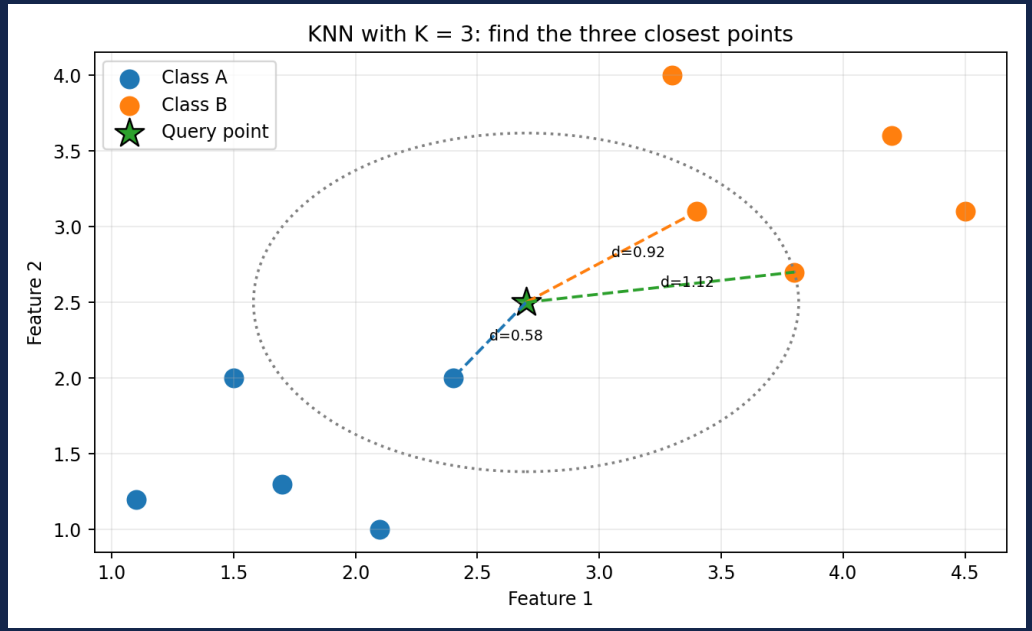
K-NEAREST NEIGHBORS (KNN)

A distance-based, instance-learning algorithm for classification and regression

Machine Learning
University Lecture



Concepts • Mathematics • Python • Evaluation

Learning outcomes

By the end of this lecture, students should be able to…

Explain the intuition behind KNN and distinguish classification from regression.

Calculate Euclidean distance and use majority voting for a new observation.

Explain why feature scaling and a suitable value of K matter.

Build, evaluate, and visualize a KNN model in Python.

Discuss computational cost, failure cases, and practical safeguards.

Where does KNN fit in machine learning?

KNN is a supervised, non-parametric, instance-based method.

Supervised learning: training examples have known target labels or values.

Classification: predict a category (e.g., spam / not spam).

Regression: predict a numeric value (e.g., house price).

Instance-based / lazy learning: little explicit model fitting; much work happens at prediction time.

Non-parametric: it does not assume a fixed functional form for the decision boundary.

Core intuition

Nearby observations in feature space are often similar.

A key caveat

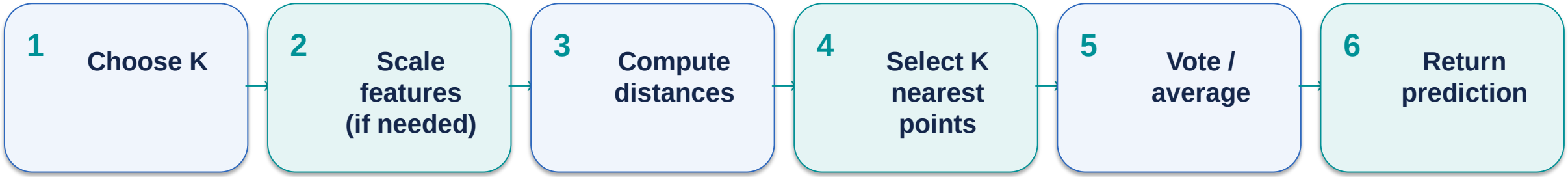
Similarity is only meaningful when features and their scales are appropriate.

Prediction

Find neighbors → combine their labels or values.

KNN: the prediction workflow

The training examples are retained and consulted for each query.



Training vs. testing

Split data into training and test sets. Fit preprocessing only on training data; at inference, KNN compares the new point with stored training observations.

Distance: how close are two observations?

For two points x and z with p features:

Euclidean distance

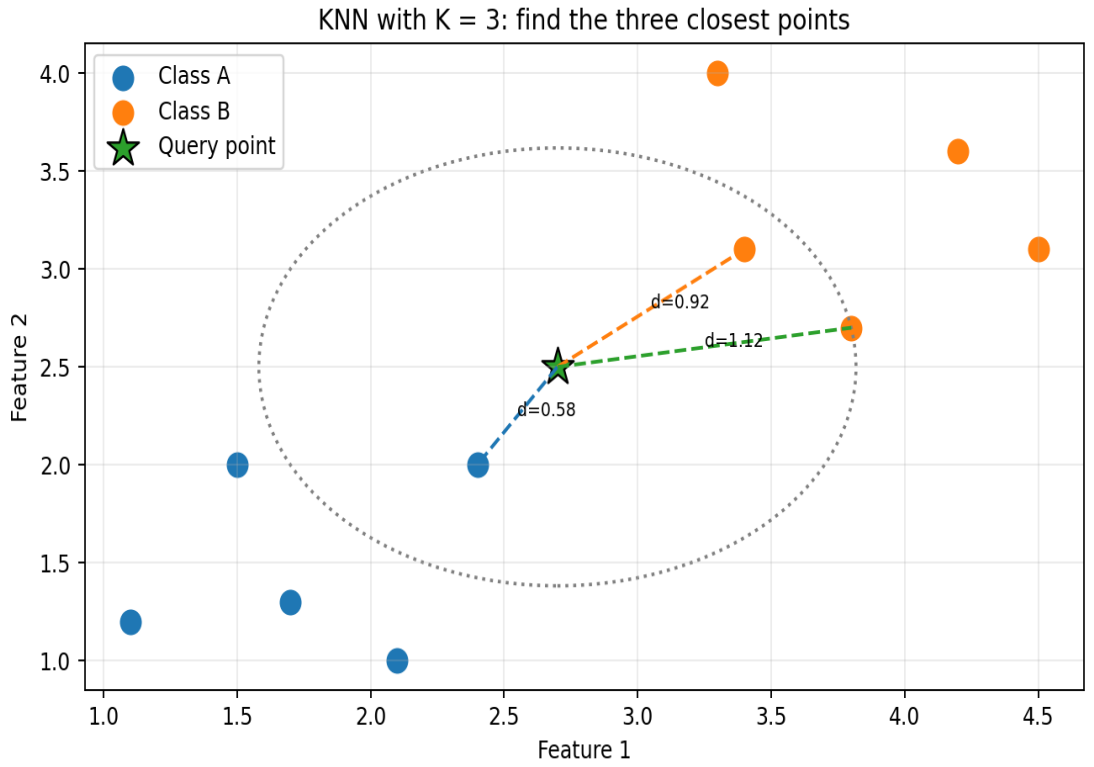
$$d(x, z) = \sqrt{[\ \Sigma_j\ (x_j - z_j)^2\ ]}$$

Straight-line distance in feature space.

Manhattan distance: sum of absolute coordinate differences.

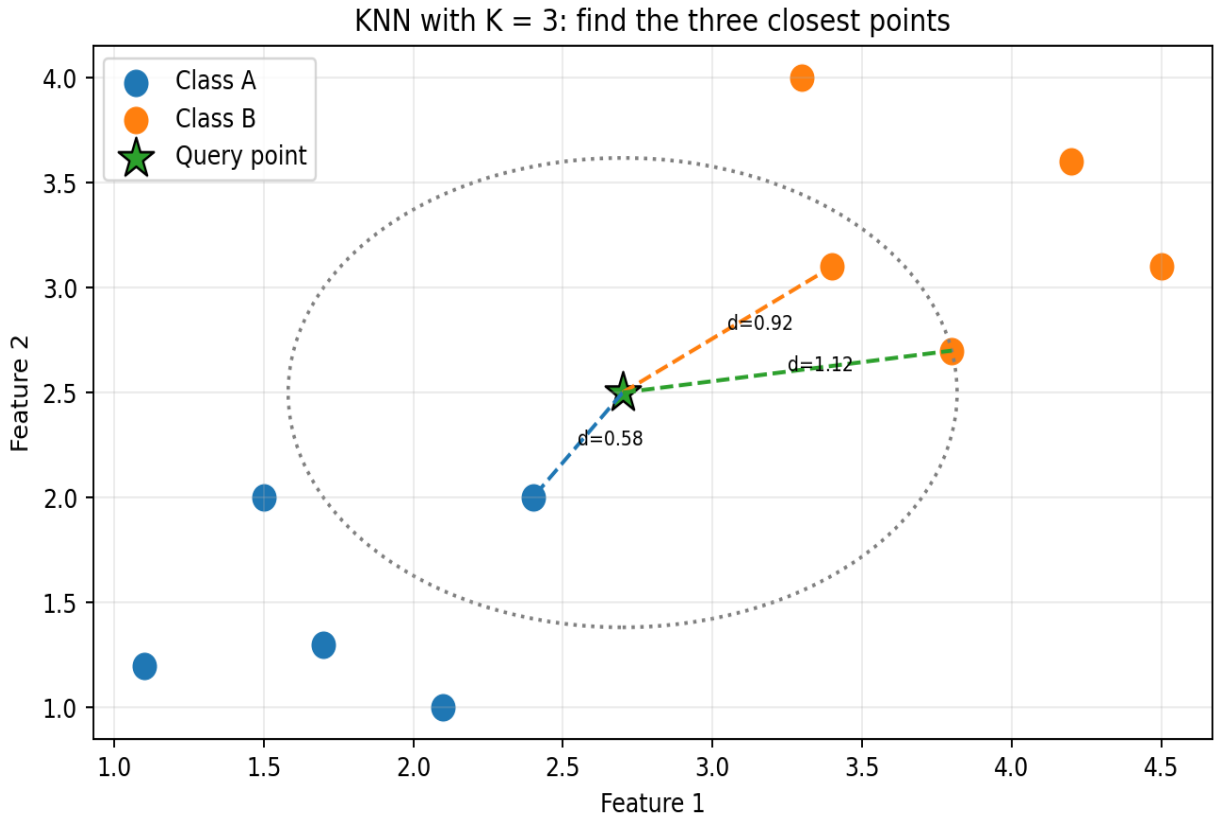
Minkowski distance generalizes several distance measures.

Distance choice should match the feature representation and problem.



Worked example: classify a query point

Illustrative two-feature data; K = 3.



Compute the distance from the query point to every labelled observation.

Sort distances from smallest to largest.

Take the three closest observations.

Count their labels; the majority label is the predicted class.

With K = 3, a 2-to-1 vote predicts the class represented twice.

Two uses of KNN

The neighborhood is the same idea; the aggregation rule changes.

KNN classification

Use majority voting across the K nearest labels. Optionally weight votes by distance.

KNN regression

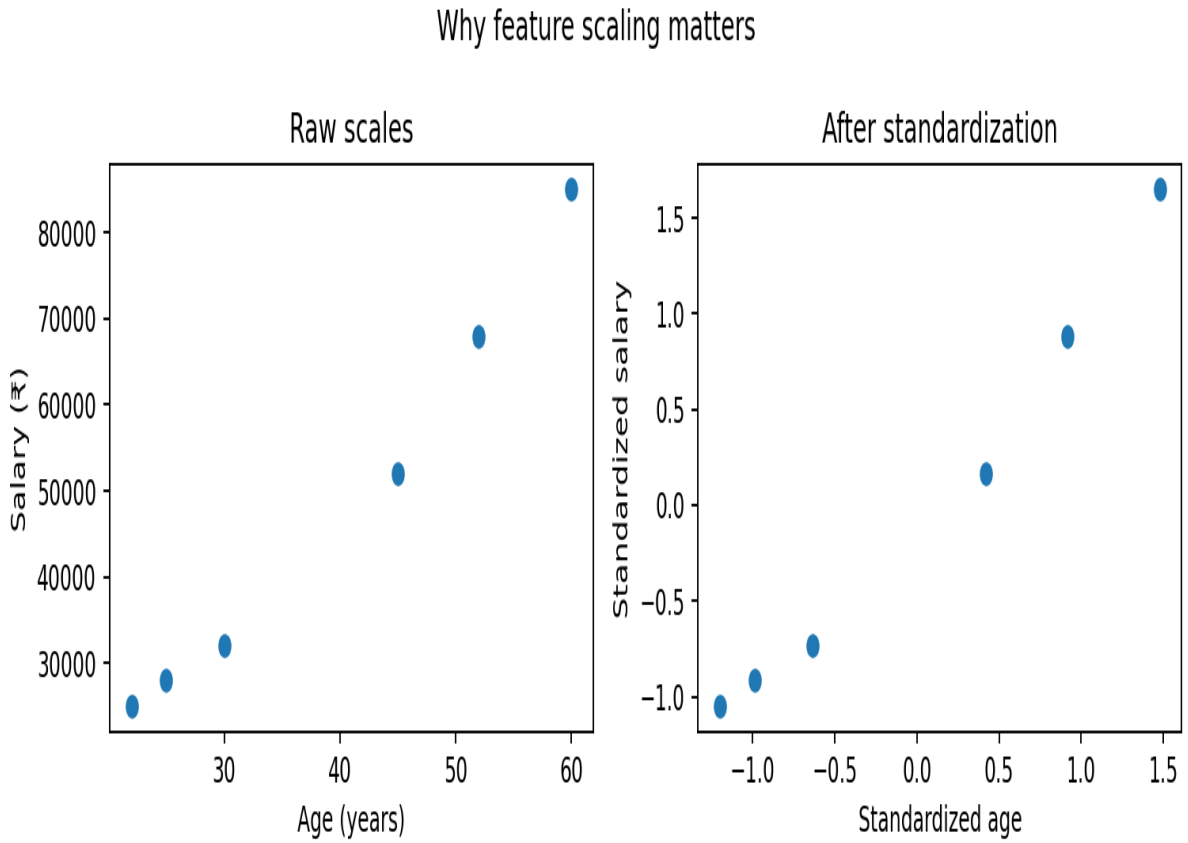
Predict by averaging (or distance-weighting) the target values of the K nearest observations.

Example

Classification: “Is this fruit an apple or an orange?”
Regression: “What is the likely selling price of this house?”

Why feature scaling is essential

A feature with a large numeric range can dominate distance.



Example: age (years) and salary (₹) have very different ranges.

Standardization: $z = (x - \mu) / \sigma$.

StandardScaler learns μ and σ from training data.

Use a Pipeline to avoid data leakage: fit scaling on training data only, then transform validation/test data.

How should we choose K?

Use a validation strategy rather than relying only on a rule of thumb.

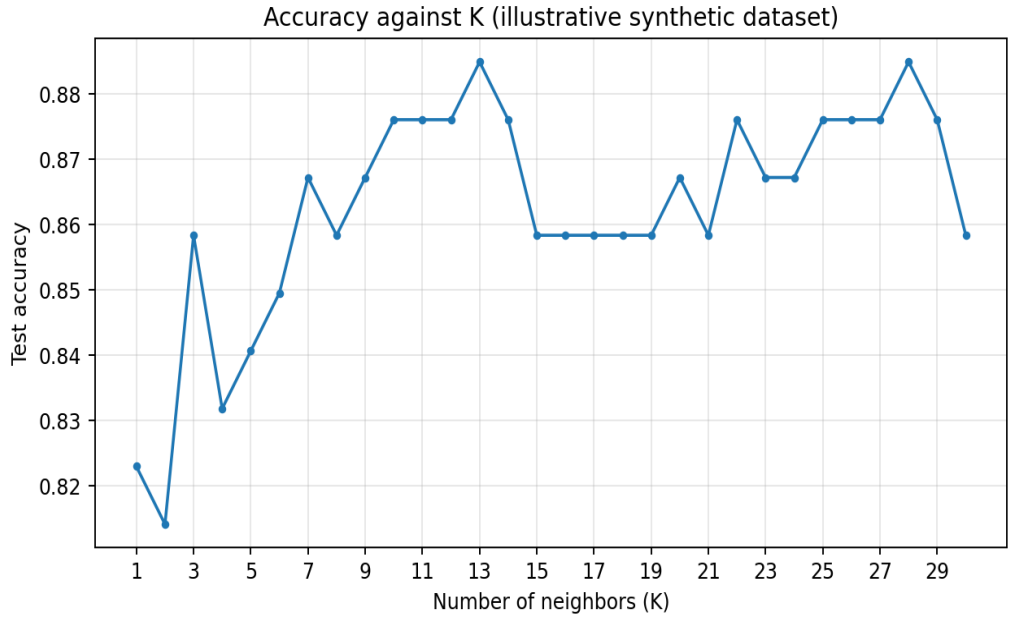
Heuristic from the notes: start near $\sqrt{n}$, where n is the number of training observations.

Compare candidate values of K using cross-validation on the training set.

Choose the metric that matches the task and class balance; accuracy is not always sufficient.

Odd K can reduce ties in binary classification, but does not guarantee that ties never occur.

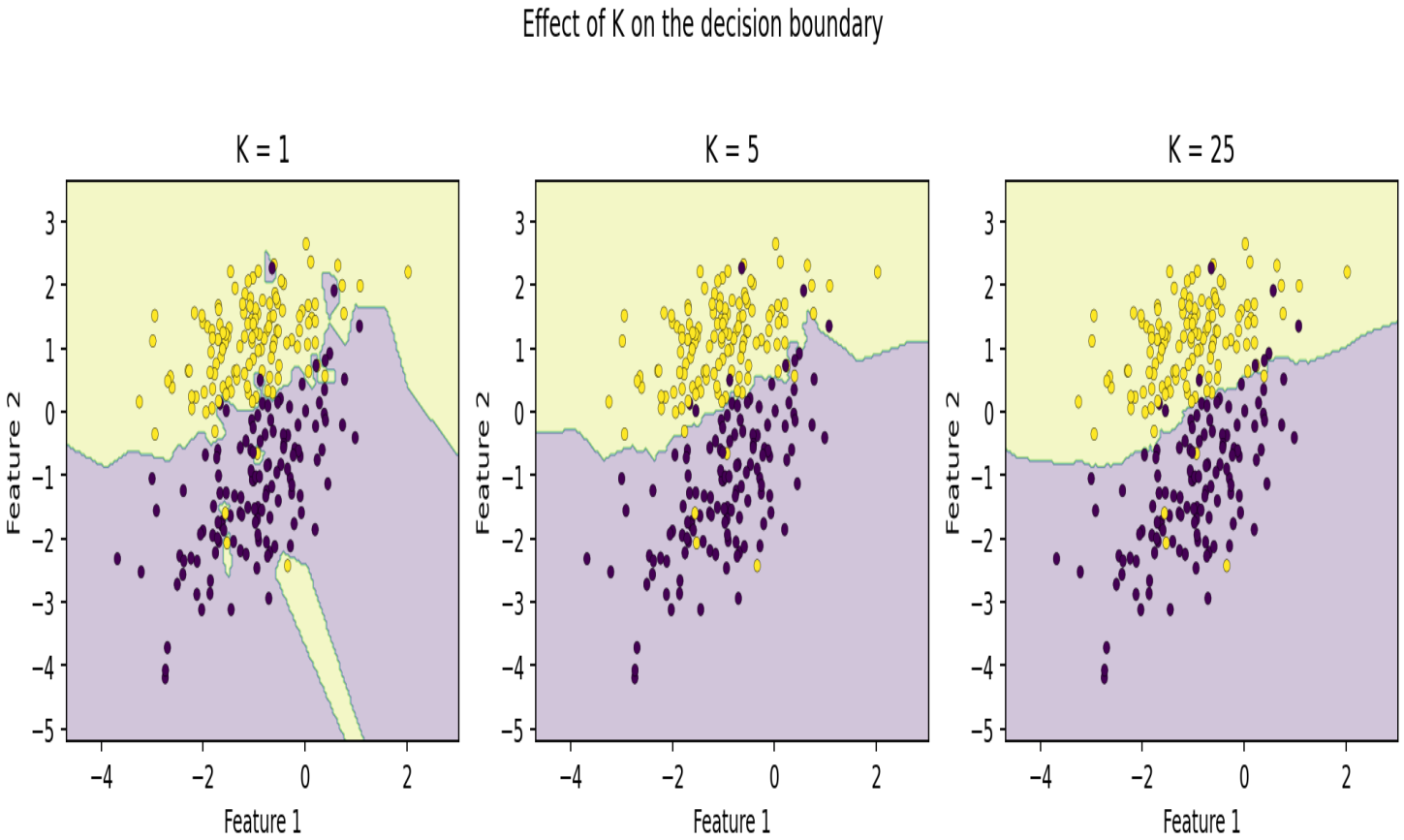
Do not select K using the final test set; reserve it for final evaluation.



Illustrative data: the best K depends on the dataset.

Decision boundaries and the bias–variance trade-off

A boundary shows which class a model predicts across feature space.



K = 1: highly flexible boundary; sensitive to noise (overfitting risk).

Larger K: smoother boundary; may ignore local structure (underfitting risk).

Choose K empirically with cross-validation.

KNN algorithm: pseudocode

A useful way to explain what the computer does for one query.

Input: labelled training set D, query x, number of neighbors K

1. For each training observation xi, compute d(x, xi)

2. Sort observations by distance

3. Keep the first K observations

4a. Classification: return the majority label

4b. Regression: return the mean target value

Output: predicted label or numeric value

Strengths and limitations

The same simplicity that makes KNN intuitive creates practical constraints.

Strengths

- Simple to explain and implement
- No strong parametric assumptions
- Can model nonlinear boundaries
- Supports classification and regression
- Useful baseline for many problems

Limitations

- Prediction can be slow on large datasets
- Stores training examples (memory cost)
- Sensitive to feature scale and irrelevant features
- Distance becomes less informative in high dimensions
- Sensitive to noise, outliers, and class imbalance

Evaluating a KNN classifier

Split data before modelling; use cross-validation for model selection.

Accuracy = correct predictions / all predictions.

Precision = TP / (TP + FP): among predicted positives, how many are correct?

Recall = TP / (TP + FN): among actual positives, how many are found?

F1-score = harmonic mean of precision and recall.

Inspect the confusion matrix, especially for imbalanced datasets.

Teaching note

For a rare positive class, a high accuracy score can hide poor recall.

A small numerical example

Query q = (3, 3); calculate Euclidean distances.

Point	Coordinates	Class	Distance to q
A	(1, 1)	Red	√8 ≈ 2.83
B	(2, 2)	Red	√2 ≈ 1.41
C	(4, 3)	Blue	1.00
D	(5, 4)	Blue	√5 ≈ 2.24
E	(3, 5)	Blue	2.00

For K = 3

Nearest: C (Blue), B (Red), E (Blue). Majority = Blue.

Prediction

The query point is classified as Blue.

Distance formula: $d = \sqrt{[(x_1 - 3)^2 + (x_2 - 3)^2]}$

Recommended scikit-learn workflow

A clean pipeline keeps preprocessing and modelling together.

```
Pipeline([
  ('scaler', StandardScaler()),
  ('knn', KNeighborsClassifier(n_neighbors=5))
])
```

Split into training and test sets.

Fit the pipeline on training data.

Use cross-validation to tune K.

Evaluate once on the held-out test set.

Report more than accuracy when class imbalance matters.

Applications and discussion

KNN is most useful when a meaningful neighborhood can be defined.

Pattern recognition and simple image-feature classification.

Recommendation or similarity-based retrieval (with suitable representations).

Medical or industrial classification as a baseline, with careful validation.

Discussion: What happens if one feature is measured in rupees and another in years?

Discussion: Why might KNN struggle with hundreds of irrelevant features?

Key takeaways

KNN predicts from the labels or values of nearby training examples.

K determines the size of the neighborhood; tune it empirically.

Scale numeric features and keep preprocessing inside the training pipeline.

Validate carefully and use appropriate metrics.

Simple to understand, but potentially expensive at prediction time.

Questions / classroom discussion